

PROGRAMAÇÃO PARA JOGOS I

Alysson Diniz
George Gomes

Problema I

Execute e analise o código 01src.py disponível no SIGAA e veja se pensou em algo semelhante

Vamos pensar nas entidades de um jogo fictício.

Imagine que tenhamos:

- um player (que se move com input do teclado)
- um inimigo terrestre, que se move de maneira automática em linha reta para a esquerda e
- um inimigo que voa, e se movimenta para a esquerda em um movimento de senoide.

Na opinião de vocês, quais classes seriam necessárias para modelar estas entidades?

Problema I

Como poderíamos alterar este código para incluir um inimigo que voasse E atirasse? E um inimigo que não voe, mas atire?

Aqui encontramos problemas de escala, duplicação de código, abstração de relações de entidades.

Problema II

Vamos para um novo problema. Imaginemos este trecho de código no método update de uma entidade de jogo fictícia.

```
if (colidiuChao(obj) && estadoRenderizacao(obj) != INVISIBLE)
{
    tocaSom(HIT_FLOOR) ;
}
```

Observe que a manutenção nesta duas linhas de código pode afetar / ser afetada pelos domínios de física, renderização e som.

Exemplo do problema no domínio de renderização

Problema III

Decorations: visível, mas **sem** interação

arbustos, detritos e outros detalhes visuais

Props: visível, mas **com** interação

caixas, árvores, etc.

Zones: invisível mas **com** interação

dispara alguma ação

Como modelar em classes uma entidade que pode ser decoration, prop, ou zone?

Problema III

Decoration

rendering

Prop

rendering

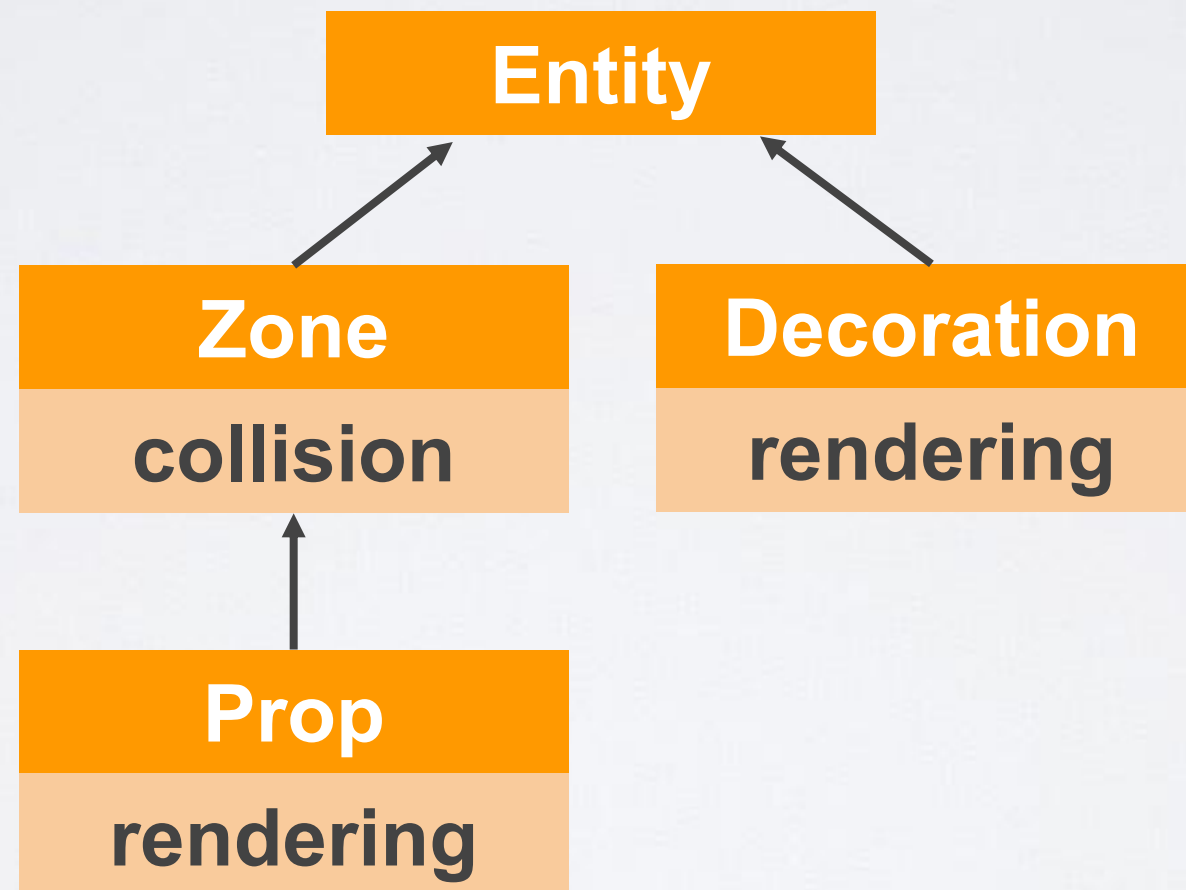
collision

Zone

collision

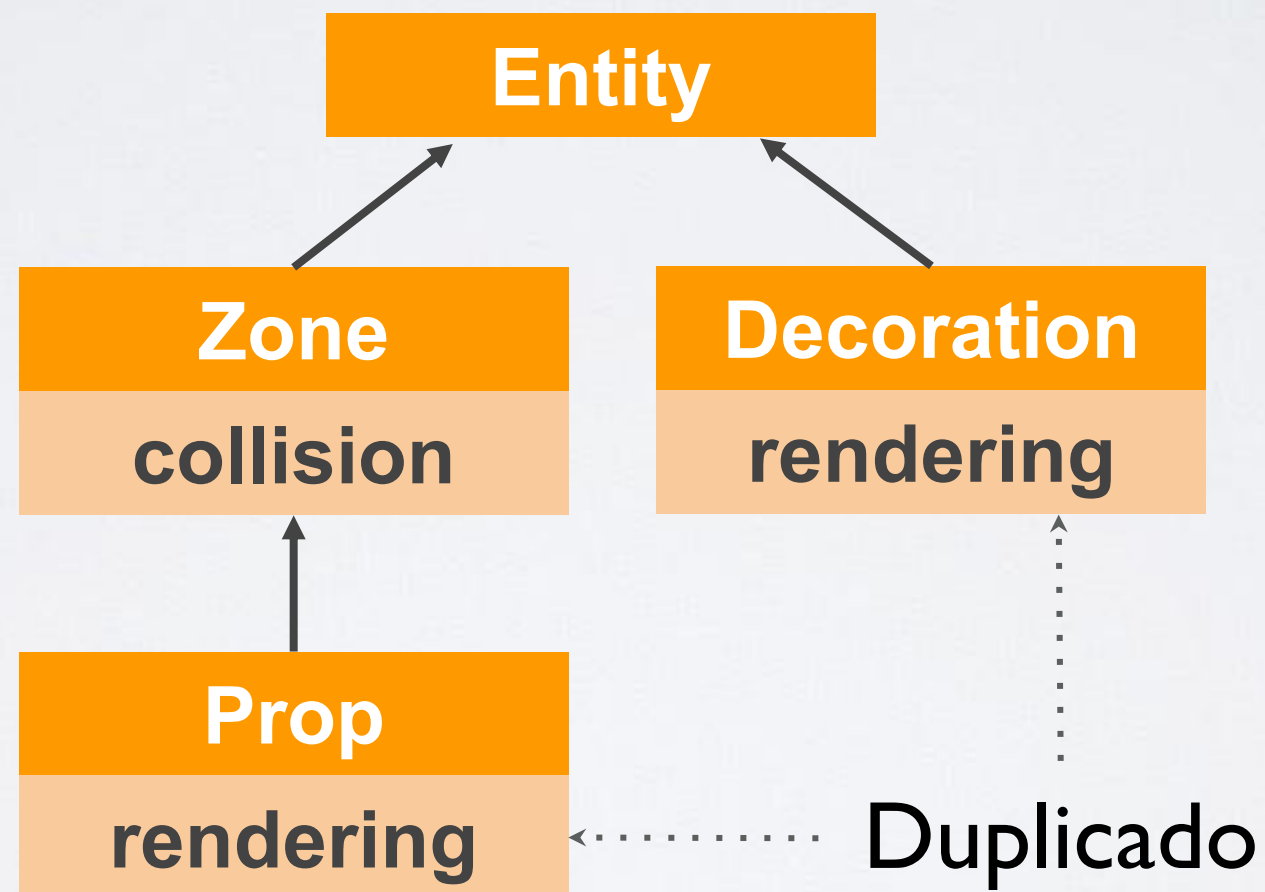
É possível resolver desta maneira, mas perceba que estamos usando 3 entidades não-relacionadas para representar itens do jogo. É um problema de abstração.

Problema III

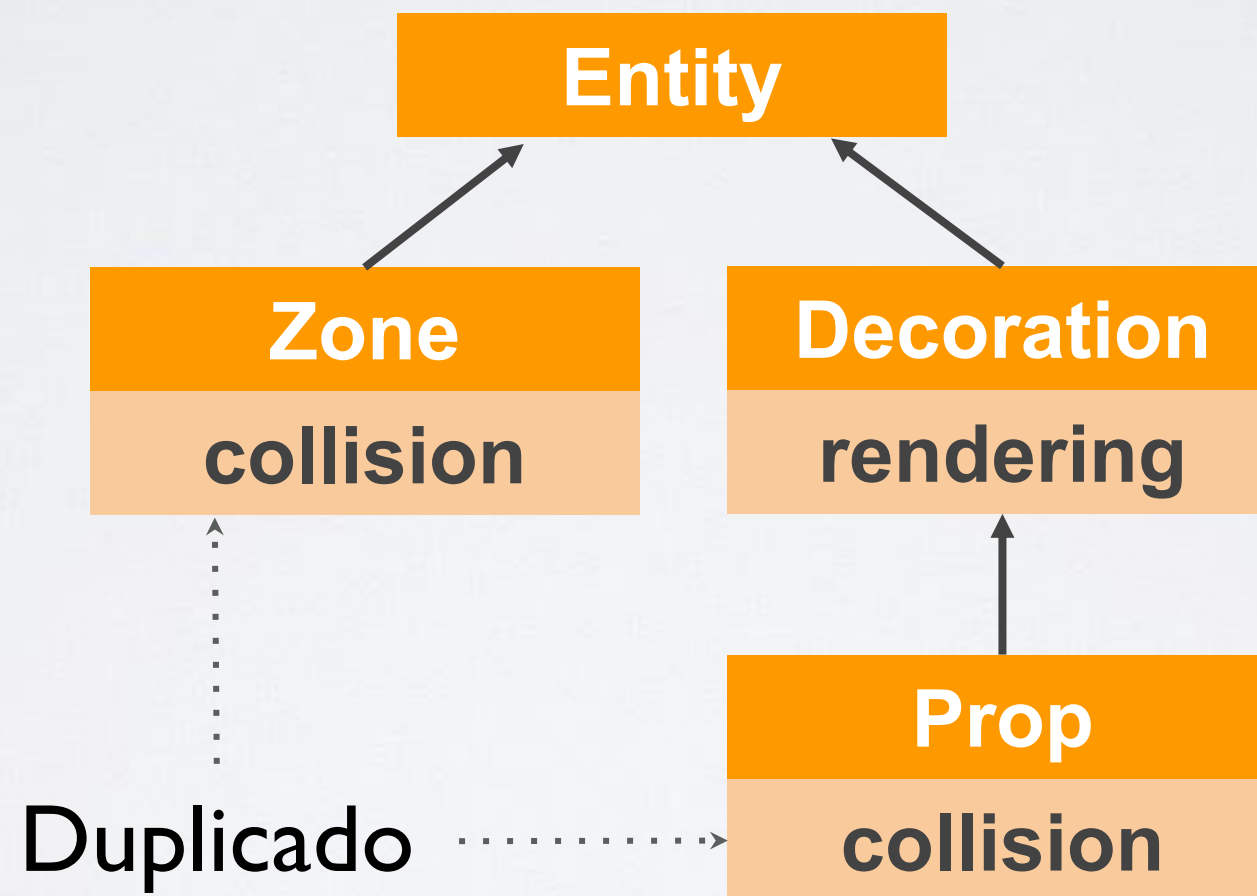


Tentando utilizar herança.

Problema III



Problema III



Como resolver o problema?

Através da criação de componentes para
cada domínio

Componentes

Entity

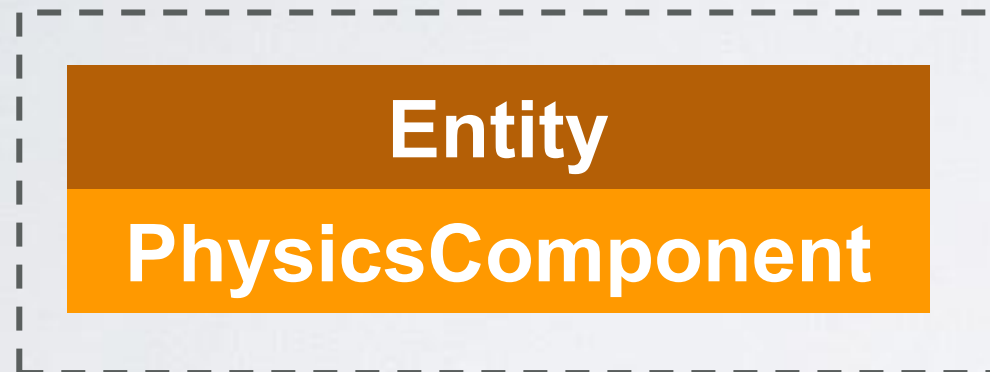
PhysicsComponent

GraphicsComponent

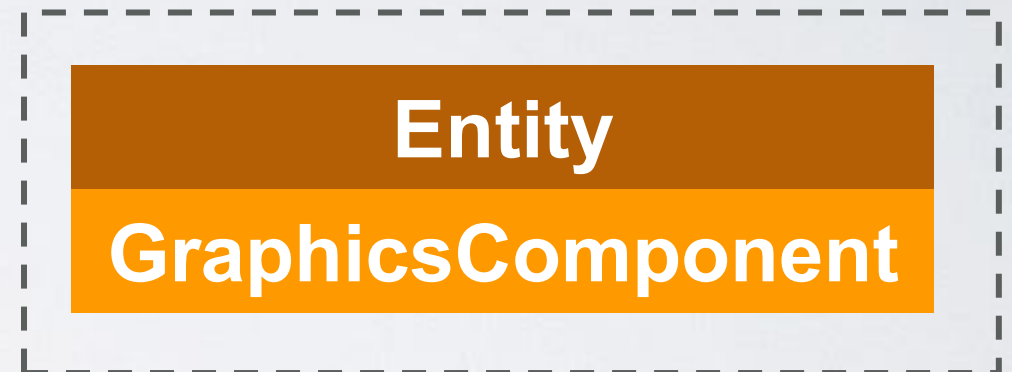
Sem duplicação de código, sem herança múltipla

Componentes

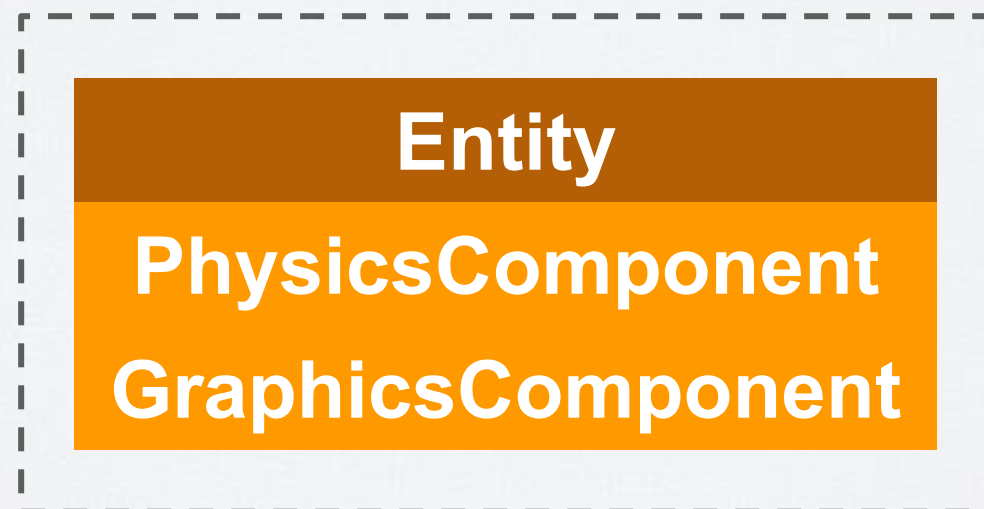
Zone



Decoration



Prop



Cada objeto será um conjunto de
componentes

Padrão de Projeto Component

Uma única entidade abrange múltiplos domínios.

Para manter os domínios isolados, o código de cada um é colocado em sua própria classe de componente. A entidade é reduzida a um simples contêiner de componentes.

Padrão de Projeto Component

Possibilita que uma entidade única lide com vários domínios de maneira *desacoplada*.

Exercício: como implementar?

Partindo do código disponibilizado para vocês, criem uma classe `RenderComponent`

- no construtor a classe deve receber a entidade a ser renderizada
- no método `draw` receba a tela na qual o jogo está sendo renderizado e chame o método `pygame.draw.circle` de maneira semelhante ao que é feito em `Player`, `Enemy` e `Flying Enemy`

Componentes

Plug-and-play para objetos

Permite criar entidades complexas
com comportamentos sofisticados
conectando diferentes componentes
e reutilizáveis

○ Padrão Componente

Uma única entidade abrange vários domínios

Mantém os domínios isolados

Cada um é colocado em seu próprio componente

A entidade é reduzida a um simples
contêiner de componentes

Referencias:

Game Programming Patterns (Decoupling Patterns)

Capítulo 14 - Component

<http://www.gameprogrammingpatterns.com/component.html>